dnb7638

Report for Single-Stage and Five-Stage Pipelined Processor

Outline

1. Introduction

2. Single-Stage Processor Design

3. Five-Stage Pipelined Processor Design

4. Execution Flow

5. File Input and Output

6. Performance Metrics and Analysis

7. Edge Cases and Error Handling

8. Conclusion

1. Introduction

The evolution of computer architecture has consistently focused on enhancing the performance, efficiency, and scalability of processors. Among the foundational designs in computer architecture are single-stage and pipelined processors, each representing a distinct approach to executing instructions.

A **single-stage processor**, also known as a single-cycle processor, processes one instruction at a time. The entire instruction—comprising fetch, decode, execute, memory access, and write-back—is completed within a single clock cycle. While straightforward in design and easy to implement, this architecture is inherently limited in performance. It requires the clock cycle duration to be long enough to accommodate the slowest instruction, leading to inefficiencies in processing.

On the other hand, a **five-stage pipelined processor** introduces overlapping execution stages, breaking down the instruction processing into five stages: Instruction Fetch (IF), Instruction Decode (ID), Execute (EX), Memory Access (MEM), and Write-Back (WB). Each stage operates concurrently on a different instruction, effectively increasing throughput. However, this parallelism introduces complexity in handling hazards:

- **Data Hazards** occur when instructions depend on the results of previous instructions.
- **Control Hazards** arise from branch instructions, which can disrupt the pipeline's flow.

In this report, we simulate and compare the performance of these two architectures using a **RISC-V instruction set**. The single-stage and pipelined processor simulators are implemented in Python, leveraging the RISC-V ISA's modular design and flexibility. Key aspects such as handling hazards, performance monitoring, and memory interactions are incorporated into the simulation.

The report aims to address the following objectives:

1. Design and implement simulators for single-stage and five-stage pipelined processors.
2. Handle pipeline hazards (RAW and control hazards) in the pipelined processor through stalling and forwarding mechanisms.
3. Measure and compare performance metrics such as **CPI (Cycles Per Instruction)**, **IPC (Instructions Per Cycle)**, and **total execution cycles** for both architectures.
4. Analyze the results to demonstrate the advantages and challenges of pipelining.
5. Propose potential enhancements to further optimize performance.

By comparing these two processor designs, this report highlights the trade-offs between simplicity and performance. The insights gained can serve as a foundation for exploring advanced processor architectures and optimization techniques.

2. Single-Stage Processor Design

The single-stage processor operates on a **single-cycle architecture**, where each instruction is fetched, decoded, executed, accesses memory (if required), and writes back the result—all within a single clock cycle. This simplicity is both its strength and its limitation, as it provides straightforward implementation but suffers from performance inefficiencies due to the fixed clock cycle duration determined by the slowest instruction.

2.1 Design Overview

The design of the single-stage processor follows a sequential flow:

1. **Instruction Fetch (IF):**
   - The processor retrieves an instruction from the instruction memory (imem.txt) at the address stored in the Program Counter (PC).
2. **Instruction Decode (ID):**
   - The fetched instruction is decoded into its components: opcode, source registers (rs1 and rs2), destination register (rd), and immediate values (if applicable).
3. **Execute (EX):**
   - Based on the decoded instruction type (e.g., R-type, I-type, J-type), the appropriate computation or operation is performed using the Arithmetic Logic Unit (ALU).
4. **Memory Access (MEM):**
   - For memory-related instructions (e.g., LW, SW), this stage interacts with the data memory (dmem.txt) to load or store data.
5. **Write-Back (WB):**
   - The result of the computation or memory access is written back to the destination register in the Register File.

2.2 Code Implementation

The implementation for the single-stage processor in the simulator is encapsulated in the SingleStageCore class. The key components of the class include:

- **State Management:**
- The processor maintains the state of the instruction being executed, including the PC, register values, and memory.
- The State object is used to track the current and next states, ensuring proper transitions between clock cycles.
- **Instruction Decoding and Execution:**
- Decoded instructions are categorized into **R-type**, **I-type**, **J-type**, **B-type**, and memory instructions. Each type has a dedicated execution logic:
- **R-Type:** Arithmetic and logical operations such as ADD, SUB, AND, OR, and XOR are executed using the ALU.
- **I-Type:** Immediate operations like ADDI, ANDI, and ORI perform computations involving immediate values.
- **J-Type and B-Type:** Branching and jump instructions modify the PC to facilitate control flow.
- **Memory Instructions:** LW (load word) and SW (store word) interact with the data memory.
- **Cycle-by-Cycle Execution:**
- Each cycle fetches a new instruction and processes it entirely within the same cycle. The step() function encapsulates the single-stage execution flow:

1. Fetch the instruction from imem.
2. Decode the instruction.
3. Execute the operation or access memory.
4. Write back results to the register file.

- **State and Register Output:**
- At the end of each cycle, the processor's state and register file are logged to output files (StateResult_SS.txt and RFResult_SS.txt) for debugging and performance analysis.

2.3 Memory and Register Interactions

1. **Instruction Memory (**imem.txt**):**

- Instructions are read sequentially based on the PC value.

- The memory is implemented as an array where each instruction occupies a 4-byte segment.

2. **Data Memory (**dmem.txt**):**

- Used for LW and SW instructions, enabling the processor to interact with external data.

3. **Register File:**

- A set of 32 registers is maintained, with x0 hardwired to 0. The RegisterFile class manages reading and writing register values.

2.4 Performance Characteristics

- **Advantages:**
- Simplicity: The single-stage processor is easy to implement and debug due to its straightforward design.
- Predictability: Each instruction completes in exactly one cycle, making execution timing deterministic.
- **Limitations:**
- **Inefficiency:** The clock cycle duration is determined by the slowest instruction, leading to wasted time for faster instructions.
- **No Overlapping Execution:** Only one instruction is executed at a time, resulting in low throughput.

2.5 Edge Case Handling

- The processor includes mechanisms to halt execution upon encountering invalid or unsupported instructions.
- The NOP (no-operation) state ensures graceful handling of empty instructions or program termination.

The single-stage processor serves as a foundation for understanding the execution flow of instructions. Its limitations, such as low throughput and inefficiency, pave the way for the introduction of pipelined architectures, which address these issues by enabling concurrent instruction processing.

3. Five-Stage Processor Design

The five-stage pipelined processor is a performance-enhanced design that divides the execution of an instruction into five stages. By overlapping the execution of multiple instructions, this architecture improves throughput and maximizes resource utilization. Each stage works concurrently on a different instruction in the pipeline, ensuring that multiple instructions are in progress at any given moment.

3.1 Design Overview

The five-stage pipeline architecture breaks the instruction execution process into the following stages:

1. **Instruction Fetch (IF):**
   - Retrieves the next instruction from the instruction memory based on the Program Counter (PC).
   - The PC is incremented by 4 unless a branch or jump modifies it.
2. **Instruction Decode (ID):**
   - Decodes the fetched instruction into its opcode, operands, and immediate values.
   - Reads source registers (rs1 and rs2) from the register file.
   - Detects hazards (e.g., RAW) and inserts stalls or forwarding as necessary.
3. **Execute (EX):**
   - Performs arithmetic and logical operations using the ALU.
   - Computes memory addresses for load and store instructions.
   - Evaluates branch conditions and calculates target addresses.
4. **Memory Access (MEM):**
   - Loads data from memory for load instructions (LW).
   - Writes data to memory for store instructions (SW).
5. **Write Back (WB):**
   - Writes results back to the destination register for all instructions that produce a result (e.g., arithmetic, load).

Each stage operates independently in the same clock cycle. A new instruction is fetched in every cycle, leading to improved instruction throughput.

3.2 Code Implementation

The implementation for the five-stage processor in the simulator is encapsulated in the FiveStageCore class. Key components include:

- **Pipeline States:**
- Each stage maintains its own state object (State_five), which includes fields for instructions, operands, ALU results, and control signals.
- States are passed from one stage to the next at the end of each clock cycle.
- **Pipeline Stages:**
- Each pipeline stage is implemented as a class:
- **Instruction Fetch (IF):** Manages fetching instructions from imem and updating the PC.
- **Instruction Decode (ID):** Decodes instructions and detects hazards.
- **Execute (EX):** Performs ALU operations and branch evaluations.
- **Memory Access (MEM):** Handles load and store instructions.
- **Write Back (WB):** Writes results back to the register file.
- **Stalling and Forwarding:**
- The processor handles **RAW hazards** (data dependencies) using a combination of stalling and forwarding:
- **Stalling:** Inserts NOPs to delay execution until dependencies are resolved.
- **Forwarding:** Directly forwards results from later stages (e.g., MEM or WB) to earlier stages (e.g., EX).
- **Control Hazards:**
- Control hazards from branches and jumps are resolved by stalling or flushing the pipeline until the branch condition is evaluated.

3.3 Memory and Register Interactions

1. **Instruction Memory (**imem.txt**):**

- Functions identically to the single-stage processor but supports concurrent accesses across stages.

2. **Data Memory (**dmem.txt**):**

- Read and write operations are managed independently in the MEM stage, enabling simultaneous execution of other stages.

3. **Register File:**

- The RegisterFile class supports multiple concurrent reads and writes, accommodating the parallel nature of pipelined execution.

3.4 Performance Characteristics

- **Advantages:**
- **Higher Throughput:** By overlapping instruction execution, the processor completes one instruction per cycle in the ideal case.
- **Resource Utilization:** Stages operate in parallel, ensuring all components are active during execution.
- **Challenges:**
- **Pipeline Hazards:** Requires sophisticated control mechanisms to handle RAW, WAR, and control hazards.
- **Pipeline Flushing:** Branch mispredictions lead to performance penalties due to flushing.

3.5 Edge Case Handling

- **Branch Prediction:** The processor stalls until the branch condition is resolved, minimizing incorrect executions.
- **Invalid Instructions:** Like the single-stage processor, the five-stage implementation halts gracefully on encountering unsupported or invalid instructions.

The five-stage processor represents a significant advancement over the single-stage architecture. While it introduces complexity in handling hazards and maintaining state consistency, it delivers substantially better throughput and resource utilization. These advantages make it the preferred choice for modern processors that prioritize performance.

4. Execution Flow

The execution flow for both the **single-stage** and **five-stage** processors describes how instructions are fetched, decoded, executed, and completed. Below, we detail the step-by-step processes for each design, highlighting their distinct execution mechanisms.

4.1 Single-Stage Processor Execution Flow

The single-stage processor executes instructions sequentially, completing all steps for one instruction before proceeding to the next.

1. **Fetch:**
   - The instruction at the address indicated by the Program Counter (PC) is fetched from the instruction memory (imem.txt).
   - The PC is incremented by 4 for the next instruction.
2. **Decode:**
   - The fetched instruction is parsed into its opcode, operands, and immediate values.
   - The register file (RegisterFile) is accessed to fetch the values of the source registers (rs1 and rs2).
3. **Execute:**
   - The ALU performs the operation specified by the opcode and function fields (e.g., ADD, SUB, XOR).
   - For memory operations (e.g., LW, SW), the effective memory address is computed.
4. **Memory Access:**
   - For load instructions (LW), data is read from the computed address in the data memory (dmem.txt).
   - For store instructions (SW), data is written to the specified address.
5. **Write Back:**
   - The result of the operation or memory access is written back to the destination register in the register file.
6. **Advance to Next Instruction:**
   - After completing all stages, the processor moves to the next instruction.

- The simulator halts execution when a HALT instruction is encountered.

4.2 Five-Stage Processor Execution Flow

The five-stage processor executes instructions concurrently in a pipelined manner, with each instruction moving through the following stages:

1. **Instruction Fetch (IF):**
- Fetches the instruction from imem.txt using the PC.
- Updates the PC to point to the next instruction unless a branch modifies it.
- The instruction is passed to the **Instruction Decode** stage in the next cycle.
2. **Instruction Decode (ID):**
- Decodes the fetched instruction into opcode, operands, and immediate values.
- Accesses the register file to fetch the values of the source registers (rs1 and rs2).
- Detects hazards (e.g., RAW) and resolves them by stalling or forwarding.
- Passes control and data signals to the **Execute** stage.
3. **Execute (EX):**
- The ALU performs operations such as addition, subtraction, logical operations, or address computation.
- Evaluates branch conditions and determines whether to modify the PC.
- Passes results to the **Memory Access** stage.
4. **Memory Access (MEM):**
- For load instructions (LW), reads data from memory.
- For store instructions (SW), writes data to memory.
- Passes the result to the **Write Back** stage.
5. **Write Back (WB):**
- Writes the result of the instruction back to the destination register in the register file.
- Marks the instruction as completed.

4.3 Key Differences Between Single-Stage and Five-Stage Execution

Aspect	Single-Stage Processor	Five-Stage Processor
Execution Model	Sequential execution, one instruction at a time.	Pipelined execution, multiple instructions in progress.
Cycle Time	One cycle per instruction.	One cycle per stage, overlapping execution.
Throughput	Low, one instruction per cycle.	High, up to one instruction per cycle after filling.
Hazard Handling	No hazards due to sequential execution.	Resolves hazards using stalling and forwarding.
Stall Penalties	N/A (no pipeline).	Occurs due to branch or RAW hazards.

4.4 Example Execution:

Suppose the following instructions are executed:

1. ADD x1, x2, x3 (R-Type)
2. LW x4, 0(x5) (I-Type)
3. SW x6, 4(x7) (S-Type)
4. BEQ x8, x9, Label (B-Type)

Single-Stage Processor:

- Each instruction is fetched, decoded, executed, and completed sequentially.
- Total cycles = 4 (one per instruction).

Five-Stage Processor:

- At cycle 1, the first instruction is fetched (IF).
- At cycle 2, the first instruction is in ID, while the second instruction is in IF.
- At cycle 3, the first instruction is in EX, the second in ID, and the third in IF.

- After cycle 5, the pipeline is full, with each stage working on a different instruction.

4.5 Performance Impact of Hazards

1. **RAW Hazards:**
- **Single-Stage:** No impact due to sequential execution.
- **Five-Stage:** Requires stalling or forwarding mechanisms.
2. **Control Hazards:**
- **Single-Stage:** Resolves before moving to the next instruction.
- **Five-Stage:** Branch mispredictions cause pipeline flushing.
3. **Structural Hazards:**
- Both architectures avoid this by using separate memory and ALU resources.

4.6 Simulator Workflow

- The simulator for both designs processes instructions from imem.txt and interacts with the dmem.txt for data operations.
- Results are stored in the submissions folder, including:
- **State Files:** StateResult_SS.txt and StateResult_FS.txt.
- **Performance Metrics:** CPI, IPC, and execution cycles.

The execution flow demonstrates the simplicity of the single-stage processor versus the complexity and efficiency of the five-stage design, emphasizing the trade-off between implementation complexity and performance.

5. File Input and Output

The simulator for both the single-stage and five-stage processors utilizes a structured approach to handle file-based input and output. This section explains the roles of input files (imem.txt and dmem.txt) and the outputs generated during simulation.

5.1 Input Files

The simulator requires two primary input files:

1. **Instruction Memory (**imem.txt**):**

- Contains the binary machine instructions for the program to be executed.
- Each instruction is represented as a 32-bit binary string.
- The instructions are read sequentially by the simulator to populate the instruction memory array.

Example Content (imem.txt):

00000000000000000000000010010011
00000000000000010000000010110011
00000000000000010000100000011100011

- Line 1: ADDI x1, x0, 0
- Line 2: ADD x2, x0, x1
- Line 3: BEQ x2, x0, 8

Usage in Code:

- The InsMem class reads the file line by line, storing the binary strings into the IMem array.

2. **Data Memory (**dmem.txt**):**

- Contains the initial state of the data memory.
- Each line corresponds to one memory location, represented as an 8-bit binary value.

- The file is used to populate the simulator's data memory array.

Example Content (dmem.txt):

00000001
00000010
00000011

- Line 1: Memory location 0 contains 0x01.
- Line 2: Memory location 1 contains 0x02.
- Line 3: Memory location 2 contains 0x03.

Usage in Code:

- The DataMem class reads the file and initializes the memory array. Unused memory is padded with 0x00.

5.2 Output Files

The simulator produces several output files for both the single-stage and five-stage cores. These files capture the state of the processor and the results of the simulation.

1. **Register File States:**

- **Single Stage:** RFResult_SS.txt
- **Five Stage:** RFResult_FS.txt
- Captures the state of all 32 registers after each cycle.
- Each line represents the binary value of a register, written in 32-bit format.

Example Content:

State of RF after executing cycle: 1
00000000000000000000000000000000
00000000000000000000000000000001
00000000000000000000000000000010
...

2. **State Files:**

- **Single Stage:** StateResult_SS.txt
- **Five Stage:** StateResult_FS.txt
- Logs the internal state of the processor after each cycle.
- For the five-stage processor, the states of each pipeline stage (IF, ID, EX, MEM, WB) are included.

Example Content:

State after executing cycle: 1

IF.PC: 4

ID.nop: False

EX.alu_op: 00

...

3. **Data Memory State:**

- **Single Stage:** SS_DMEMResult.txt
- **Five Stage:** FS_DMEMResult.txt
- Contains the state of the data memory after the program finishes execution.
- Each line represents an 8-bit binary value corresponding to a memory location.

Example Content:

00000001

00000010

00000011

4. **Performance Metrics:**

- PerformanceMetrics_Result.txt
- Captures the key performance metrics for both cores, including:

- Total cycles
- Total instructions executed
- CPI (Cycles per Instruction)
- IPC (Instructions per Cycle)

Example Content:

Single Stage Core Performance Metrics:

Number of Cycles taken: 10

Number of Instructions: 5

CPI: 2.0

IPC: 0.5

Five Stage Core Performance Metrics:

Number of Cycles taken: 8

Number of Instructions: 5

CPI: 1.6

IPC: 0.625

5.3 Directory Structure

The input and output files are organized into directories to maintain clarity:

- **Input Files:**
- code/imem.txt
- code/dmem.txt
- **Output Files:**
- submissions/StateResult_SS.txt
- submissions/StateResult_FS.txt
- submissions/RFResult_SS.txt
- submissions/RFResult_FS.txt
- submissions/PerformanceMetrics_Result.txt
- submissions/SS_DMEMResult.txt

- submissions/FS_DMEMResult.txt

5.4 File Handling in Code

- The InsMem and DataMem classes are responsible for reading and writing the input and output memory files.
- The main simulation loop ensures that the results are written to the output files at the appropriate stages of execution.

This structured input and output system ensures the simulator runs efficiently, with all necessary information logged for detailed analysis.

6. Performance Metrics and Analysis

Performance metrics are essential for evaluating the efficiency of the single-stage and five-stage pipelined processors. The simulator implements performance monitors to calculate and report key metrics, providing a comparative analysis of both architectures.

6.1 Performance Metrics Overview

The following metrics are computed for both processors:

1. **Cycles Per Instruction (CPI):**

- CPI represents the average number of clock cycles required to execute one instruction.
- Formula:

CPI = Total Cycles Taken / Total Instructions Executed

2. **Instructions Per Cycle (IPC):**

- IPC indicates how many instructions are executed per clock cycle.
- Formula:

IPC = Total Instructions Executed / Total Cycles Taken

3. **Total Execution Cycles:**

- The total number of clock cycles required to complete program execution.

4. **Instruction Count:**

- Total number of instructions executed, including HALT instructions.

6.2 Single-Stage Core Metrics

- In the single-stage processor, each instruction is executed sequentially. Therefore, each instruction occupies the entire pipeline for its execution.
- **Expected Behavior:**
- Higher CPI due to sequential execution.
- Lower IPC as only one instruction executes in one cycle.
- **Example Metrics Output:**

Single Stage Core Performance Metrics:

Number of Cycles Taken: 20

Number of Instructions Executed: 10

CPI: 20 / 10 = 2.0

IPC: 10 / 20 = 0.5

6.3 Five-Stage Core Metrics

- In the five-stage pipelined processor, multiple instructions are executed simultaneously across different stages.
- **Pipeline Hazards and Their Impact:**
- RAW (Read After Write) hazards are managed using data forwarding.
- Control hazards (e.g., branch instructions) may introduce stalls.
- **Expected Behavior:**
- Lower CPI due to instruction overlap in the pipeline.
- Higher IPC as multiple instructions progress through the stages in parallel.
- **Example Metrics Output:**

Five Stage Core Performance Metrics:

Number of Cycles Taken: 12

Number of Instructions Executed: 10

CPI: 12 / 10 = 1.2

IPC: 10 / 12 = 0.833

6.4 Comparative Analysis

Metric	**Single-Stage Processor**	**Five-Stage Processor**
Total Cycles Taken	Higher (e.g., 20)	Lower (e.g., 12)
Instructions Executed	Equal (e.g., 10)	Equal (e.g., 10)
CPI	Higher (e.g., 2.0)	Lower (e.g., 1.2)
IPC	Lower (e.g., 0.5)	Higher (e.g., 0.833)

6.5 Code Implementation

1. **Performance Calculation:**

- In the Performance_metrics function, metrics are computed using cycle and instruction counts.
- The results are logged into a file PerformanceMetrics_Result.txt.

Example Code:

```
def Performance_metrics(opFilePath: str, ss: SingleStageCore, fs: FiveStageCore):
    ss_metrics = [
        "Single Stage Core Performance Metrics:",
        f"Number of Cycles Taken: {ss.cycle}",
        f"Number of Instructions Executed: {ss.inst}",
        f"CPI: {ss.cycle / ss.inst:.2f}",
        f"IPC: {ss.inst / ss.cycle:.2f}",
    ]

    fs_metrics = [
        "Five Stage Core Performance Metrics:",
        f"Number of Cycles Taken: {fs.cycle}",
        f"Number of Instructions Executed: {fs.num_instr}",
        f"CPI: {fs.cycle / fs.num_instr:.2f}",
        f"IPC: {fs.num_instr / fs.cycle:.2f}",
    ]

    with open(opFilePath + os.sep + "PerformanceMetrics_Result.txt", "w") as f:
        f.write("\n".join(ss_metrics) + "\n\n" + "\n".join(fs_metrics))
```

2. **Terminal Output:**

- The metrics are displayed on the terminal for quick verification during execution.

Example Terminal Output:

Single Stage Core Performance Metrics:
Number of Cycles Taken: 20
Number of Instructions Executed: 10
CPI: 20 / 10 = 2.0
IPC: 10 / 20 = 0.5

Five Stage Core Performance Metrics:
Number of Cycles Taken: 12
Number of Instructions Executed: 10
CPI: 12 / 10 = 1.2
IPC: 10 / 12 = 0.833

6.6 Insights and Observations

1. **Pipelining Efficiency:**

- The five-stage processor demonstrates improved performance due to reduced CPI and increased IPC.

2. **Impact of Hazards:**

- RAW and control hazards can reduce the efficiency of the pipeline, but effective hazard handling (e.g., forwarding) minimizes their impact.

3. **Execution Time:**

- The pipelined processor achieves faster execution due to its ability to handle multiple instructions concurrently.

7. Edge Cases and Error Handling

In a robust simulator design, handling edge cases and potential errors is critical to ensure accuracy and reliability during execution. Both the single-stage and five-stage cores are equipped with mechanisms to handle various scenarios that could disrupt normal processing.

7.1 Edge Cases in Single-Stage Processor

1. **Unsupported Instructions:**

- The single-stage processor must only handle the specified subset of RISC-V instructions (e.g., ADD, SUB, LW, SW, etc.).
- **Handling:**
- Any unsupported instruction results in the processor entering a “halt” state.
- Example: An unknown opcode triggers the nop flag in the instruction fetch state.
- **Code Snippet:**

```
# HALT for unsupported instructions
else:
   self.state.IF["nop"] = True
```

2. **Out-of-Bounds Memory Access:**

- The processor must ensure that all memory reads and writes stay within the allocated memory boundaries.
- **Handling:**
- Attempting to access memory outside dmem or imem results in an exception or is ignored to maintain stability.

3. **Immediate Sign Extension:**

- Proper handling of sign extension for immediate values is necessary to ensure correct arithmetic operations for both positive and negative values.
- **Example:**

```
def sign_extend(val, sign_bit):
```

```
    if (val & (1 << sign_bit)) != 0:
        val = val - (1 << (sign_bit + 1))
    return val
```

7.2 Edge Cases in Five-Stage Processor

1. **Pipeline Hazards:**

- **RAW (Read After Write):**
- Occurs when an instruction depends on the result of a previous instruction that has not yet written to a register.
- **Handling:**
- Data forwarding is used to fetch the value directly from the EX or MEM stages instead of waiting for the WB stage.
- Example:

```
def detect_hazard(self, rs):
    if rs == self.state.MEM.write_reg_addr and self.state.MEM.read_mem == 0:
        return 2  # EX to EX forwarding
    elif rs == self.state.WB.write_reg_addr and self.state.WB.write_enable:
        return 1  # MEM to EX forwarding
```

- **Control Hazards (Branch Instructions):**
- Branches and jumps can disrupt the pipeline by invalidating instructions already in progress.
- **Handling:**
- A “stall” mechanism delays further instruction fetches until the correct branch target is known.
- Example: The hazard_nop flag ensures no further instructions are fetched.

2. **Pipeline Flush:**

- During control hazards, invalid instructions in the pipeline must be flushed to prevent incorrect results.
- **Implementation:**
- A flush is implemented by setting the nop flag in the pipeline stages.

3. **Instruction Alignment:**

- Ensures instructions are fetched in aligned 32-bit segments.
- **Handling:**
- Misaligned memory access results in errors or a processor halt.

7.3 Error Logging

1. **File Not Found:**

- If the imem.txt or dmem.txt files are missing, the simulator halts with a clear error message.
- **Code Example:**

```
if not os.path.exists(ioDir + os.sep + "imem.txt"):
    raise FileNotFoundError(f"'imem.txt' not found in {ioDir}")
```

2. **Invalid Opcodes:**

- Unrecognized opcodes are logged for debugging purposes and result in a halt.

3. **Boundary Condition Violations:**

- Memory reads/writes outside the allocated memory space are gracefully ignored or logged.

7.4 Robust Testing

To ensure the simulator handles all edge cases effectively, the following testing methodology is employed:

- **Boundary Tests:**
- Accessing the first and last addresses of imem and dmem.
- **Instruction Coverage:**
- Testing all supported opcodes, including both valid and invalid inputs.
- **Hazard Handling Validation:**
- Simulating RAW and control hazards to verify the correctness of forwarding and stalls.
- **Exception Handling:**
- Testing the simulator's behavior with missing or malformed input files.

7.5 Insights and Observations

1. **Simulator Stability:**
- Proper handling of edge cases ensures stability, preventing crashes during execution.
2. **Pipeline Reliability:**
- Effective hazard detection and resolution techniques, such as forwarding and stalling, enhance the reliability of the pipelined processor.
3. **User Feedback:**
- Clear error messages and logs guide users in resolving issues, improving the overall usability of the simulator.

This comprehensive error-handling framework ensures that the simulator performs robustly across a wide range of scenarios, providing accurate results and insightful feedback to users.

8. Conclusion

The single-stage and five-stage pipelined processor simulators were developed to model the execution of RISC-V instructions, providing insights into the functioning of modern processor architectures. The implementation successfully demonstrates the core principles of instruction execution, including arithmetic operations, memory access, and control flow, while accounting for key challenges such as data and control hazards.

The **single-stage processor** offers simplicity in design and execution but suffers from performance bottlenecks due to the sequential processing of instructions. In contrast, the **five-stage pipelined processor** introduces parallelism, achieving significant performance gains through overlapping instruction execution. The inclusion of hazard detection and resolution mechanisms, such as forwarding and stalling, further enhances its effectiveness.

By measuring key metrics like CPI (cycles per instruction), IPC (instructions per cycle), and total execution cycles, the report highlights the efficiency of the pipelined design over the single-stage implementation. The pipelined approach consistently achieves a lower CPI and higher IPC, making it better suited for high-performance applications.

This simulator serves as a valuable tool for understanding processor architecture fundamentals, with its modular design allowing for easy extension and experimentation. The analysis underscores the importance of pipelining in improving processor efficiency and lays the foundation for exploring more advanced concepts in computer architecture.